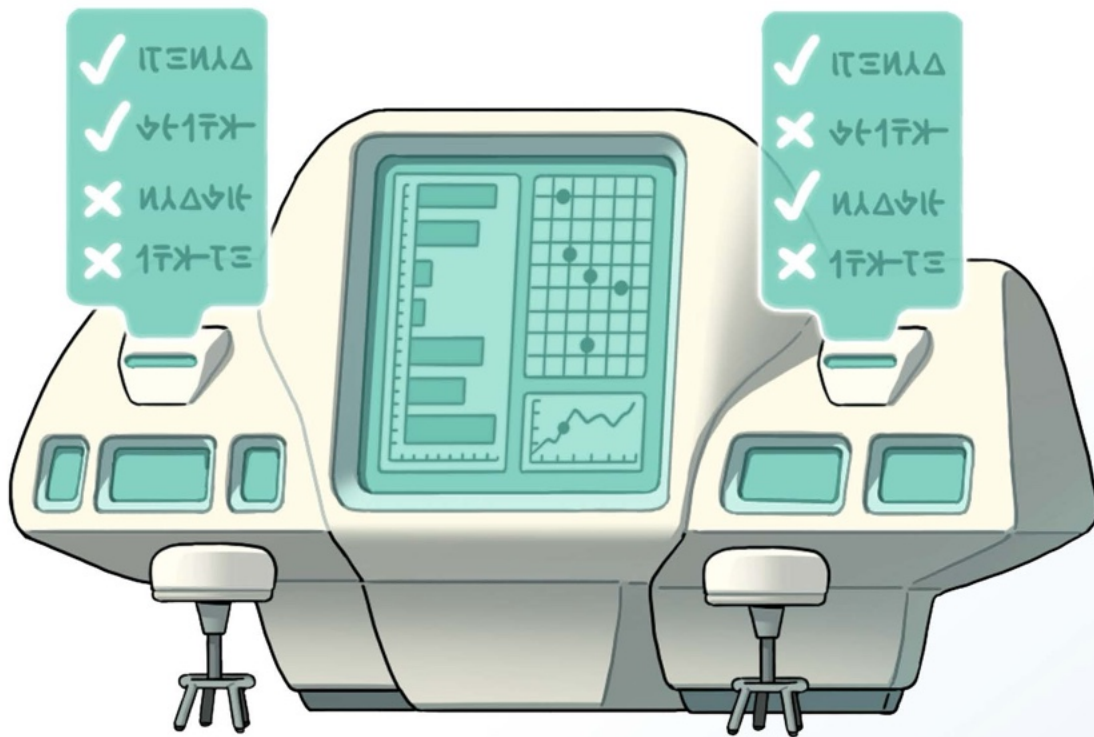




The Mac admin's guide to switching from Jamf to Fleet

Everything you know about Jamf, mapped to how Fleet does it.

Contents

- 4 Introduction
- 5 Why Jamf admins consider switching
- 5 Fleet Free vs. Fleet Premium
- 6 Chapter 1 - Jamf to Fleet: the terminology translation
- 8 Chapter 2 - Enrollment
- 9 Chapter 3 - Configuration profiles
- 9 What stays the same
- 9 What's different
- 10 Jamf variables → Fleet variables
- 11 Managing configuration profiles with GitOps
(the Fleet-native way)
- 12 Chapter 4 - Grouping devices
- 12 Fleets: The equivalent of Jamf Sites
- 13 Labels: The equivalent of Jamf Smart Groups
- 14 Migrating your Smart Group logic
- 14 Chapter 5 - Compliance checks
- 15 A more direct compliance approach
- 15 When a check fails: automatic remediation



- 16 Chapter 6 - Software deployment: packages, self service, and patching
 - 16 Package deployment: Jamf Policies vs. Fleet Software
 - 17 Fleet-maintained apps: pre-packaged software
 - 17 Migrating your Jamf software catalog
- 18 Chapter 7 - osquery for beginners
 - 19 The osquery mental model
 - 20 Your first 15 reports
- 22 Chapter 8 - GitOps: your new version control workflow
 - 22 What GitOps means in practice
 - 23 Getting started with Fleet GitOps
 - 24 GitOps vs. UI: when to use each
- 24 Chapter 9 - The migration plan
 - 25 How Apple's MDM migration works
 - 25 Phase 1 - Set up Fleet and deploy agent
 - 26 Phase 2 - Build Fleet configuration
 - 26 Phase 3 - IT team migration
 - 26 Phase 4 - Department-by-department rollout
 - 27 Phase 5 - Decommission Jamf

Thank you to the following people for their review of this guide: [Andrea Pepper](#), [Kitzy](#), [Henry Stamerjohann](#), [Harry Ravazzolo](#), Steven Palmesano, [Erin Miska](#), [Dan Gordon](#)



Introduction

This guide is for you if you've spent years mastering Jamf Pro, know your way around policies, Smart Groups, and Extension Attributes, and are now evaluating Fleet as an alternative, or if you've already decided to migrate and want to get up to speed fast.

This guide doesn't start from zero. It assumes you know MDM and focuses on translating what you already know in Jamf's language and mapping it to Fleet.

Who this guide is for

- | | |
|---|---|
| <ul style="list-style-type: none">✓ Mac admins with 1-10+ years of Jamf Pro experience✓ IT engineers evaluating Fleet as a Jamf replacement✓ Teams that have already decided to migrate and want a fast ramp✓ Teams managing complex environments who've hit the limits of what Jamf can see and control | <ul style="list-style-type: none">✗ Not for: teams new to MDM |
|---|---|

Why Jamf admins consider switching

If you're still weighing the decision, these are the most common reasons admins move from Jamf to Fleet:

- **Real-time device data.** Fleet queries device state in near real time.
- **Simpler compliance workflows.** Real-time visibility into installed software and patch status means fewer fire drills at audit time.
- **One API.** Fleet has a single REST API and a CLI (fleetctl). No juggling multiple API versions.
- **Infrastructure as code.** Fleet configuration lives in Git. Changes are reviewed, versioned, and deployed automatically.
- **Cross-platform.** Manage macOS, Windows, and Linux from the same console with the same workflow.
- **Self-hosting option.** Fleet can be cloud-hosted or self-hosted, so you keep full control of your data.

Fleet Free vs. Fleet Premium

Fleet's core is open source under the MIT license. Technically, you can self-host Fleet for free with unlimited devices and users.

But if you're migrating from Jamf Pro, you're migrating to Fleet Premium. The open-source version is a powerful visibility tool, but the management capabilities you rely on as a Mac admin (MDM profiles, ADE enrollment, FileVault key escrow) are Premium-only.

Fleet Free (open source) gives you real-time visibility into device state via live SQL queries. It's great for security teams that need to see what's happening without changing anything. While it offers full management for Linux, its relationship with macOS and Windows is essentially read-only.

Fleet Premium is the full Jamf replacement. You get Apple MDM, ADE/ABM enrollment, VPP app distribution, FileVault key escrow, and native profile management for macOS, iOS, and Windows. It also upgrades your workflow with GitOps (infrastructure-as-code).

Chapter 1 - Jamf to Fleet: the terminology translation

Jamf and Fleet use different terminology. Here's how the concepts line up.

Core concept mapping

Concept	In Jamf Pro	In Fleet
Agent	Jamf Management Framework / jamf binary	fleetd (powered by osquery)
Multi-tenancy	Sites	Fleets
Device data collection	Recon (scheduled query)	osquery (real-time)
Device record	Inventory record	Host Details page
Custom data collection	Extension Attributes	Live SQL query (via osquery)
Device grouping	Smart Groups / Static Groups	Dynamic Labels / Manual Labels
Device search	Advanced search/inventory	Live SQL query (via osquery)
User-to-device mapping	User Record	IdP mapping
Device enrollment	PreStage Enrollment	Setup Experience
Remote execution (software deployment, scripts)	Policy	Software / Scripts

OS patching	Policy / MDM command / Blueprints	OS Updates (via DDM)
Patch management (updating third-party apps)	Policy or App Installers (App Catalog)	Fleet Maintained Applications (FMA)
Compliance pass/fail checks	Smart Group membership	Policies (osquery-based, real-time)
Logging	Change management logs	Audit logs
Log destinations	Splunk/Elastic (manual config)	Splunk, Snowflake, Kinesis, Lambda (native)
API access	Multiple APIs (Classic + Jamf Pro)	Fleet REST API + fleetctl CLI



Chapter 2 - Enrollment

If you've set up Jamf PreStage Enrollment, the foundation is familiar: both Jamf and Fleet ride on Apple's Automated Device Enrollment (ADE) through Apple Business Manager (ABM). Each tool configures the enrollment experience differently on top of that shared layer, and the pieces don't map one-to-one. This chapter walks through the enrollment process step by step, so you can see where Fleet's Setup Experience lines up with Jamf's PreStage Enrollment, and where it differs.

Step	Jamf	Fleet
1. MDM server in ABM	Jamf registered in ABM	Fleet registered in ABM
2. Device assignment	Assign devices to PreStage in Jamf UI	Assign devices to Fleet in ABM (or auto-assign by ABM instance)
3. Enrollment trigger	Device first boot → ADE → Jamf MDM profile pushed	Device first boot → ADE → Fleet MDM profile pushed
4. Welcome screen	PreStage: Setup Assistant screen config	Fleet Setup Experience - configurable skip steps
5. Software at enrollment	Policies scoped to 'Enrollment Complete' trigger	Software packages assigned to Fleet deploy on enrollment
6. SSO / IdP login	Jamf Connect (additional license required)	Platform SSO (no separate Fleet license)



Switching MDMs doesn't mean wiping devices. Apple's native MDM migration moves them over in place, with no data loss. Chapter 9 covers how it works, along with the full migration plan.

Chapter 3 - Configuration profiles

Configuration profiles work the same way in Fleet as in Jamf. They're Apple's mechanism, mobileconfig and JSON declaration files for macOS/iOS, CSPs for Windows, and Fleet deploys them using the same underlying MDM commands. Your existing profiles will work in Fleet.

What stays the same

- **Mobileconfig file format:** identical. Export from Jamf, unsign them, import to Fleet.
- **Profile signing:** Fleet supports signed profiles using Apple developer certificates. (Note that signed Jamf profiles can not be imported.)
- **Profile categories:** Restrictions, Wi-Fi, VPN, certificates, passcode, etc. - all supported.
- **iOS/iPadOS profiles:** Same format, same deployment.
- **Profile removal behavior:** supervised devices allow MDM-initiated removal, same as Jamf.

What's different

Feature	Jamf Pro	Fleet
Profile UI	Built-in profile builder with GUI forms	Upload standard mobileconfig profiles directly (no proprietary GUI layer)
Profile scope targeting	Smart Group scope + exclusion	Fleet assignment + Label targeting
Profile versioning	No built-in version history	Full Git history when using GitOps mode
Payload variables	Jamf variables (\$COMPUTERNAME, \$USERNAME, etc.)	Fleet variables (supported, different syntax)
Profile types	mobileconfig, DDM support requires Jamf SSO and Blueprint usage	mobileconfig, JSON for DDM declarations

Profile conflict detection	Limited — conflicts visible only after deploy	Validation available via CI in GitOps workflow
Profile status tracking	Device record shows profile install status	Fleet UI shows per-device profile status with failure details
Windows CSPs	Requires separate Jamf School or Jamf Pro Windows	Native in Fleet — same workflow as macOS

Jamf variables → Fleet variables

If your Jamf profiles use built-in variables (like \$COMPUTERNAME or \$SERIAL), you'll need to update these for Fleet. Here's the mapping:

Jamf variable	what it resolves to	Fleet equivalent
\$COMPUTERNAME	Device hostname	\$FLEET_VAR_HOST_HOSTNAME
\$SERIAL	Device serial number	\$FLEET_VAR_HOST_HARDWARE_SERIAL
\$UDID	Device UDID	\$FLEET_VAR_HOST_UUID
\$USERNAME	Enrolled user's username	\$FLEET_VAR_HOST_END_USER_IDP_USERNAME
\$FULLNAME	Enrolled user's full name	\$FLEET_VAR_HOST_END_USER_IDP_FULL_NAME
\$EMAIL	Enrolled user's email	\$FLEET_VAR_HOST_END_USER_EMAIL_IDP

Managing configuration profiles with GitOps (the Fleet-native way)

In Jamf, you manage configuration profiles through the UI. In Fleet, you can still do this, but the preferred approach is GitOps: profiles live in a Git repository as mobileconfig files, and changes deploy automatically when merged.

A Fleet GitOps repo structure for profiles looks like:

```
fleet/  
  lib/  
    macos/  
      configuration-profiles/  
        disk-encryption.mobileconfig  
        filevault-escrow.mobileconfig  
        wifi-corp.mobileconfig  
        ...
```

When a PR is merged to main, GitHub Actions (or any CI tool) runs `fleetctl dry-run` to catch errors, then `fleetctl apply`, and Fleet deploys the updated profiles and declarations to all devices in that Fleet. Every change has a commit history, a reviewer, and a timestamp.

More about Fleet GitOps in Chapter 8.

Chapter 4 - Grouping devices

Managing devices requires balancing where a device "lives" and what criteria it fits into. Fleet uses two distinct constructs - fleets and labels - to handle these requirements. Understanding how these map to Jamf Sites and Smart Groups is essential for admins making the transition.

Fleets: The equivalent of Jamf Sites

In Fleet, a "fleet" is a scoped environment. Like a Jamf Site, it provides administrative isolation (RBAC), but it goes further by acting as the primary container for configuration, so you can avoid duplicating work.

Some common Fleet use cases:

Fleet	Who's in it	Key strategy
Testing	IT, Early Adopters, Beta Testers	First to receive OS updates, new scripts, and agent versions to "smoke test" changes.
Production	General Staff (Sales, HR, Finance, Marketing)	The stable baseline. Receives updates only after they are vetted in the Testing fleet.
Kiosk / Shared	Lab Macs, Conference Rooms	Locked-down, no user data, auto-wipe logic.
Mobile devices	Employee iPhones & iPads	Separated because mobile MDM payloads differ significantly from macOS workstation configurations

Labels: The equivalent of Jamf Smart Groups

While fleets define the environment and baseline settings, Labels handle the specific targeting logic within those environments. They are essentially saved search queries that categorize devices in real-time based on their specific attributes.

The fundamental difference between smart groups and labels is the engine. Jamf relies on periodic inventory reports that may or may not be up to date. Fleet Labels are evaluated via osquery, enabling real-time membership updates based on live device telemetry.

Some common Label use cases:

Label	Targeting purpose	Example query logic
Intel Macs	Deploying x86_64 vs. ARM64 software packages.	<code>cpu_brand LIKE '%Intel%'</code>
Developer Tools	Scoping specialized configs only to machines with Xcode or VS Code.	<code>SELECT 1 FROM apps WHERE name = 'Xcode.app'</code>
Virtual Machines	Excluding VMs from hardware-specific firmware updates.	<code>hardware_model LIKE '%Virtual%'</code>
Large Storage	Identifying devices capable of hosting heavy media caches.	<code>disk_size_bytes > 5000000000000 (500GB+)</code>
High Performance	Targeting Macs with 32GB+ RAM for pro-app optimizations.	<code>physical_memory > 32000000000</code>
Laptop vs. desktop	Applying specific power management profiles (Clamshell mode vs. Always On).	<code>system_info WHERE hardware_model LIKE 'MacBook%'</code>

Migrating your Smart Group logic

When migrating from Jamf, you'll want to audit your Smart Groups and decide for each one: should this become a Fleet, a Label, or a Policy?

Smart Group type	what it does	Migrate to
Org unit/department	Groups devices by business unit	Dynamic Label (based on IdP mapping)
OS version targeting	Target devices on a specific macOS version	Dynamic Label
App installed check	Devices with/without a specific app	Dynamic Label or Policy
Compliance check	Devices meeting/failing a security requirement	Policy
Hardware criteria	RAM, disk, model identifier	Dynamic Label
Enrollment status	Recently enrolled, supervised, etc.	Built-in Label (Fleet provides these automatically)

Chapter 5 - Compliance checks

It's important to remember that "policy" means something different in Fleet than it does in Jamf. In Jamf, a policy executes software installs and scripts on a trigger. In Fleet, a policy is a compliance check - a pass/fail test of whether a device meets a condition.

In Jamf, you build compliance checks with Smart Groups: if a device meets criteria X, it lands in a group, and group membership tells you it's out of compliance. It works, but it's indirect - compliance is inferred from membership, not directly tested.

Fleet tests it directly. Each Fleet policy is an osquery SQL query that runs as an always-on pass/fail check. If the query returns a row, the device passes. If it returns nothing, the device fails. No group membership to maintain, no inference - just a direct answer to "Is this device compliant?"

A more direct compliance approach

Scenario	Jamf approach	Fleet approach
OS version compliance	Smart Group criteria: 'Operating System is not 15.x'	<code>SELECT 1 FROM os_version WHERE major >= 15;</code>
Required app installed	Smart Group: 'Application Title is not CrowdStrike.app'	<code>SELECT 1 FROM apps WHERE name = 'CrowdStrike.app';</code>
Firewall enabled	EA script checks <code>/usr/libexec/ApplicationFirewall/socketfilterfw</code>	<code>SELECT 1 FROM alf WHERE global_state >= 1;</code>
Screen lock timeout	Configuration Profile enforces timeout (not a compliance check)	<code>SELECT 1 FROM preferences WHERE domain='com.apple.screensaver' AND key='idleTime' AND CAST(value AS INTEGER) <= 300;</code>

When a check fails: automatic remediation

When a device is out of compliance, Fleet can also fix it automatically. Here's what happens the moment a policy fails:

- **Auto-install software:** If a policy fails because a required app is missing, Fleet can automatically install it.
- **Auto-run scripts:** If a policy fails because a setting is wrong, Fleet can run a remediation script on the failing device.
- **Webhook alerts:** Policy failures trigger webhooks to Slack, PagerDuty, Jira, or any endpoint.
- **Ticket creation:** Fleet can create Jira or Zendesk tickets automatically when devices fail critical policies.

Chapter 6 - Software deployment: packages, self service, and patching

Software deployment is where Mac admins spend a lot of their time, and where Jamf has historically been strong. Fleet's approach is equivalent in capability and, for many workflows, simpler to maintain.

Package deployment: Jamf Policies vs. Fleet Software

Feature	Jamf	Fleet
Package format	PKG, DMG (via policy)	PKG, EXE, MSI, DEB, RPM
Deployment trigger	Policy trigger (enrollment, recurring, event, Self Service)	Install via Setup Experience, Self Service, or Policy-triggered auto-install
Install script	Pre/Post-install scripts in package or policy	Pre/Post-install scripts per package
Version tracking	Jamf tracks installed version from inventory, limited to known install paths	Fleet tracks installed version via osquery (live, not scheduled) across the host
Patch management	Jamf Patch Management (add-on)	Automatic updates via policy + software enforcement
App catalog	Patch sources limited; custom packages manual	Fleet-maintained apps (pre-packaged, auto-update), plus custom packages
Self Service app	Jamf "Self Service" App	Fleet "Desktop" App
Cross-platform	Mac-only	Mac, Windows, Linux (same workflow, same UI)



Fleet-maintained apps: pre-packaged software

Fleet maintains a curated library of pre-packaged, auto-updated applications, similar to Homebrew Cask, but managed through your MDM. These apps are tested, packaged correctly, and updated by Fleet's team when new versions are released. You don't write install scripts. You just add the app to your Fleet's software list.



Examples from the Fleet-maintained app library: Google Chrome, Firefox, Zoom, Slack, 1Password, Visual Studio Code, CrowdStrike Falcon, SentinelOne, Cloudflare WARP, and dozens more.

Migrating your Jamf software catalog

1. Export a list of your current Jamf-managed packages and Self Service items.
2. Check if the apps exist in Fleet's maintained library. These apps are patched and updated by Fleet automatically, requiring zero packaging effort from you.
3. For apps not in the library, leverage your existing AutoPkg workflows. Fleet supports AutoPkg, allowing you to automate the download, packaging, and uploading of software directly into your fleets.

Chapter 7 - osquery for beginners

Jamf's Extension Attributes (EAs) are custom scripts that collect data and report it back to Jamf, where it's stored in the device record. EAs are flexible - you can script almost any value you want - but they return data on the inventory cycle, so what you're looking at is as fresh as the last check-in.

Fleet's osquery approach is different. Instead of storing a value from the last collection, Fleet queries the device live and returns the answer in real time. The tradeoff: osquery reads from a defined set of system tables rather than arbitrary scripts, so it's less about scripting freedom and more about getting a true answer right now.

Extension Attribute → osquery translation

Check if Location Services are enabled

Jamf extension attribute:

```
#!/bin/bash
uuid=$(system_profiler SPHardwareDataType | awk -F ": " ' /
Hardware UUID/ {print $2}')
domain="/var/db/locationd/Library/Preferences/ByHost/
com.apple.locationd.${uuid}"
plist="${domain}.plist"
if [[ -f "$plist" ]]; then
    status=$(/usr/bin/defaults read "$domain"
LocationServicesEnabled 2>/dev/null)
    if [[ "$status" == "1" ]]; then
        result="Enabled"
    else
        result="Disabled"
    fi
else
    result="Unavailable"
fi
echo "<result>${result}</result>"
```

Fleet osquery:

```
SELECT enabled FROM location_services;
```

The osquery version runs in milliseconds, across all devices simultaneously.

The osquery mental model

osquery exposes operating system data as SQL tables. Every table represents something about the device, and there are over 300 of them. You can JOIN them like a database.

Table	What it contains
system_info	Hostname, hardware model, CPU, RAM, serial number
os_version	OS name, major/minor/patch version, build
users	All local accounts - username, UID, admin status, shell
apps (macOS)	All installed .app bundles - name, version, path
processes	All running processes - PID, name, path, parent, user
listening_ports	All open TCP/UDP ports with associated process
disk_encryption	Disk encryption status per volume
certificates	All certificates in the keychain - expiry, subject, issuer
launchd	All launch agents and daemons - name, path, status
chrome_extensions	All Chrome extensions installed (by user)
crashes	Recent application crashes
last	Recent logins - user, time, source IP

Learn more: osquery table reference

- The complete osquery table schema is at: <https://fleetdm.com/tables>
- Fleet's Standard Query Library includes 100+ pre-written queries for common IT and security use cases. Browse and import them at: <https://fleetdm.com/queries>
- Fleet's AI-assisted policy descriptions can generate the SQL for common compliance checks from a plain-English description of what you want to verify.

Your first 15 reports

These queries answer the questions Jamf admins most commonly write Extension Attributes for.

1. App version audit

```
SELECT name, bundle_short_version, path FROM apps WHERE  
name LIKE 'Falcon%';
```

2. Local admin audit

```
SELECT username, uid, gid FROM users WHERE uid >= 500  
AND gid = 80 AND username NOT IN ('itadmin', 'root');
```

3. Chrome extension versions

```
SELECT name, version, uid, browser_type FROM  
chrome_extensions WHERE name = '1Password' ORDER BY  
version;
```

4. Check for expired certificates

```
SELECT common_name, subject, not_valid_after FROM  
certificates WHERE not_valid_after <  
strftime('%s','now','+30 days');
```

5. Connected USB devices

```
SELECT vendor, model, serial FROM usb_devices;
```

6. Check for SSH/Remote Management

```
SELECT remote_login FROM sharing_preferences;
```

7. Audit Gatekeeper status

```
SELECT assessments_enabled, dev_id_enabled FROM gatekeeper;
```

8. SIP (System Integrity Protection) audit

```
SELECT config_flag, enabled FROM sip_config;
```

9. Unexpected network connections

```
SELECT p.name, l.address, l.port FROM processes p JOIN listening_ports  
l ON p.pid = l.pid WHERE l.port NOT IN (80, 443, 22) ORDER BY l.port;
```

10. FileVault status

```
SELECT name, uuid, encrypted FROM disk_encryption;
```

11. macOS version

```
SELECT name, version, build FROM os_version;
```

12. Uptime / last reboot

```
SELECT datetime(last_reboot, 'unixepoch') AS last_reboot FROM uptime;
```

13. Installed RAM

```
SELECT ROUND(physical_memory / 1073741824.0, 1) AS ram_gb FROM  
system_info;
```

14. Battery cycle count

```
SELECT cycle_count, health, condition FROM battery;
```

15. Startup items (persistence check)

```
SELECT name, path, source, status FROM startup_items;
```

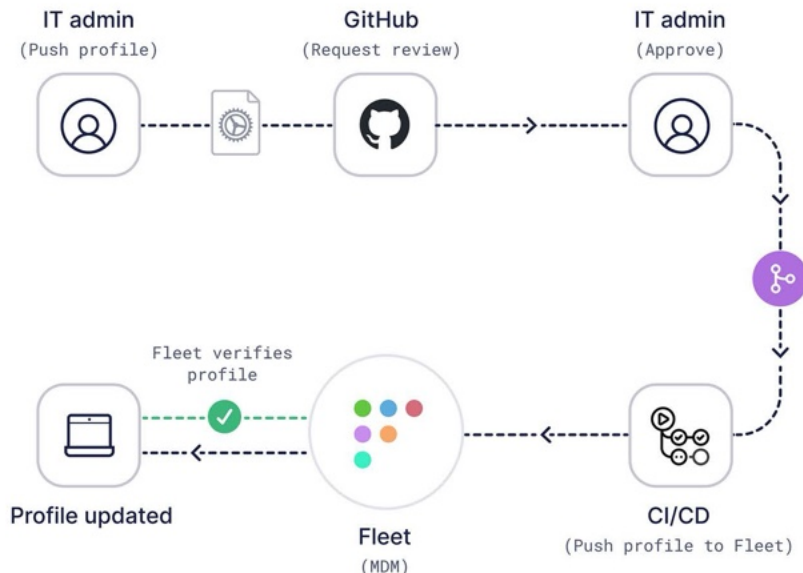
Chapter 8 - GitOps: your new version control workflow

If you've used Jamf for years, you may have experienced this: a policy change goes out, something breaks, and you have to manually reconstruct what the configuration looked like before the change. Jamf doesn't have undo or a change history. Nor does it have peer review.

Fleet's GitOps mode solves all of this by moving your MDM configuration into Git, where every change is versioned, reviewed, and reversible.

What GitOps means in practice

- Your entire Fleet configuration (fleets, profiles, policies, software, labels, agent options) lives in a Git repository as YAML and mobileconfig files.
- To make a change, you open a pull request. A teammate reviews it. Automated checks validate the config. When merged, it deploys automatically.
- To roll back a bad change: revert the commit. Fleet re-deploys the previous config within minutes.
- The Git history is your audit log: every change ever made, by whom, when, with a message explaining why.



Getting started with Fleet GitOps

You can generate a starter GitOps repo structure by running `fleetctl new`. Here's what a Fleet GitOps repo looks like:

Fleet GitOps repository structure

```

it-and-security
├── default.yml
├── fleets
│   ├── personal-mobile-devices.yml
│   └── workstations.yml
├── labels
│   ├── apple-silicon-macos-hosts.yml
│   ├── arm-based-windows-hosts.yml
│   ├── debian-based-linux-hosts.yml
│   └── x86-based-windows-hosts.yml
├── platforms
│   ├── all
│   │   ├── icons
│   │   ├── policies
│   │   └── reports
│   ├── android
│   │   ├── configuration-profiles
│   │   └── managed-app-configurations
│   ├── ios
│   │   ├── configuration-profiles
│   │   └── declaration-profiles
│   ├── ipados
│   │   ├── configuration-profiles
│   │   └── declaration-profiles
│   ├── linux
│   │   ├── policies
│   │   ├── reports
│   │   ├── scripts
│   │   └── software
│   ├── macos
│   │   ├── commands
│   │   ├── configuration-profiles
│   │   ├── declaration-profiles
│   │   ├── enrollment-profiles
│   │   │   └── automatic-enrollment.dep.json
│   │   ├── policies
│   │   │   └── all-software-updates-installed.yml
│   │   ├── reports
│   │   ├── scripts
│   │   └── software
│   └── windows
│       ├── configuration-profiles
│       ├── policies
│       ├── reports
│       ├── scripts
│       └── software
└── README.md
    
```

You can learn more about Fleet GitOps in the [Modern endpoint management: Managing devices as code](#) whitepaper and the [Fleet GitOps documentation](#).

GitOps vs. UI: when to use each

Scenario	recommended approach
Initial setup and exploration	Fleet UI - faster to experiment
Production configuration	GitOps - all changes reviewed and versioned
Compliance evidence for auditors	GitOps - Git history proves what was deployed and when
Onboarding a new IT team member	GitOps - they read the repo, not tribal knowledge
Initial setup and exploration	Fleet UI - faster to experiment
Multi-team / multi-environment	GitOps - structure by environment (staging/prod branches, or separate repos)

Chapter 9 - The migration plan

Here's a proven sequence for migrating from Jamf to Fleet. You do not need to migrate all at once or cut over immediately. Run Fleet alongside Jamf for a period before migrating any devices.



How Apple's MDM migration works

A common concern when switching MDMs is device disruption. As of macOS 26, Apple's native MDM migration moves a device from one MDM to another without wiping it, without the user losing data, and in many cases without IT touching the device at all.

Two requirements:

- Devices must be enrolled in their current MDM via Automated Device Enrollment (ADE).
- Devices must be running macOS 26 or later.

The high-level flow:

1. ABM requests an MDM migration for a managed device.
2. macOS notifies the user that their device management settings are being updated.
3. The user clicks Enroll, and the device unenrolls from Jamf and enrolls in Fleet in a single step.
4. Fleet's enrollment profile is applied. Profiles, software, and policies begin deploying.

No wipe. No FileVault recovery key regeneration. No loss of installed software.

The phases below put this into practice: deploy Fleet alongside Jamf first, build your configuration in parallel, then migrate in cohorts using this flow.

Phase 1 - Set up Fleet and deploy agent

At this stage, Jamf is still managing your devices. You're just adding visibility.

1. Deploy Fleet server (Cloud or self-hosted).
2. Connect Fleet to your IdP (Okta, Azure AD, Google Workspace).
3. Use Jamf to deploy fleetd (the Fleet agent) to all managed devices. Devices appear in Fleet without any MDM enrollment.
4. Verify Fleet shows all your devices with correct inventory data.
5. Run live queries to audit your current environment - find misconfigurations, identify your baseline.

Phase 2 - Build Fleet configuration

Still no migration yet - you're building the parallel environment.

1. Set up Fleets to match your Jamf Sites structure.
2. Recreate configuration profiles in Fleet (export from Jamf, import to Fleet - most work as-is).
3. Rebuild your most-used Extension Attributes as osquery queries. Most map directly, but some that rely on custom scripting may need rethinking - since osquery reads from system tables rather than running arbitrary scripts.
4. Rebuild your Jamf software deployment policies as Fleet Software - Fleet-maintained apps first, then custom packages (use AutoPkg).
5. Rebuild your Jamf script policies as Fleet Scripts.
6. Rebuild your Jamf Smart Groups (that check a compliance condition) as Fleet Policies.
7. Configure GitOps if using (set up repo, CI/CD pipeline, test on a branch).

Phase 3 - IT team migration

The first cohort is always IT's own devices. You can fix problems quickly, and you won't get calls from frustrated users. Migrating in cohorts also means you catch a profile conflict on a handful of machines, not across the whole fleet at once.

1. Migrate IT team devices using the Apple MDM migration flow.
2. Verify all profiles apply correctly. Test software deployment.
3. Run all policies - fix any gaps in your Fleet configuration.
4. Run parallel for 1-2 weeks: compare Fleet data to Jamf data to verify accuracy.

Phase 4 - Department-by-department rollout

Expand one team at a time. Communicate the change to each department in advance.

1. Notify the department: "Your device management is being upgraded. You'll see a notification to enroll - just click Accept."
2. Trigger MDM migration for the cohort.
3. Monitor enrollment completion rate in Fleet.
4. Follow up with non-enrolled devices (offline devices, devices that need manual intervention).
5. Confirm all software, profiles, and policies are working before moving to the next cohort.

Phase 5 - Decommission Jamf

1. Verify 100% of devices are enrolled in Fleet and have expected profiles/policies.
2. Run Jamf and Fleet in parallel for 30 days (final validation period).
3. Cancel the Jamf subscription - coordinate with the renewal date to avoid double payment.
4. Archive Jamf configuration exports for reference.
5. Remove the Jamf agent from all devices. (Fleet provides a [script](#) for this.)

Migration readiness checklist

Before migrating your first production cohort:

- ☐ Fleet server deployed and accessible
- ☐ ABM connected to Fleet MDM
- ☐ IdP connected (SSO for Fleet UI + Platform SSO for devices)
- ☐ All fleets created and configured
- ☐ All configuration profiles tested on IT devices
- ☐ Software catalog built (Fleet-maintained + custom packages)
- ☐ Top compliance policies passing on IT devices
- ☐ GitOps repo live (or manual workflow documented)
- ☐ Escalation path documented (what if a device fails to migrate?)
- ☐ User communication template ready
- ☐ Rollback plan documented (though rarely needed)

Ready to make the switch?

[Join a workshop](#)

